

CRUD operations using InMemory Database in ASP.NET Core Web API.

1. What is InMemory Database in ASP.NET Core?

The **InMemory database provider** is a lightweight, temporary data store provided by EF Core. It **does not require SQL Server** or any external database engine.

When to use InMemory DB

- Unit testing and integration testing.
- Rapid prototyping.
- Learning EF Core.
- No need for migrations.

Limitations

- Data is **not persisted** once application restarts.
 - Not suitable for production.
 - Cannot test SQL behaviors (constraints, relationships, raw SQL, etc.).
-

2. Project Setup

Step 1: Create ASP.NET Core Web API project

```
dotnet new webapi -n InMemoryApiDemo
```

Step 2: Install EF Core InMemory package

```
dotnet add package Microsoft.EntityFrameworkCore.InMemory
```

3. Create the Model

Example: Product

```
public class Product
{
    public int Id { get; set; }    // PK
```

```
public string Name { get; set; }  
public decimal Price { get; set; }  
}
```

4. Create the DbContext

```
public class AppDbContext : DbContext  
{  
    public AppDbContext(DbContextOptions<AppDbContext> options)  
        : base(options)  
    {  
    }  
  
    public DbSet<Product> Products { get; set; }  
}
```

5. Register InMemory Database in Program.cs

```
builder.Services.AddDbContext<AppDbContext>(options =>  
{  
    options.UseInMemoryDatabase("InMemoryDb");  
});
```

Important: No migrations needed.

6. Create ProductsController (CRUD)

```
[ApiController]  
[Route("api/[controller]")]  
public class ProductsController : ControllerBase
```

```

{
    private readonly AppDbContext _context;

    public ProductsController(AppDbContext context)
    {
        _context = context;
    }

    // CREATE
    [HttpPost]
    public async Task<ActionResult> CreateProduct(Product product)
    {
        _context.Products.Add(product);
        await _context.SaveChangesAsync();
        return CreatedAtAction(nameof(GetProductById), new { id = product.Id }, product);
    }

    // READ ALL
    [HttpGet]
    public async Task<ActionResult<IEnumerable<Product>>> GetAll()
    {
        return await _context.Products.ToListAsync();
    }

    // READ BY ID
    [HttpGet("{id}")]

```

```
public async Task<ActionResult<Product>> GetProductById(int id)
{
    var product = await _context.Products.FindAsync(id);

    if (product == null)
        return NotFound();

    return product;
}

// UPDATE
[HttpPut("{id}")]
public async Task<ActionResult> UpdateProduct(int id, Product updatedProduct)
{
    var product = await _context.Products.FindAsync(id);

    if (product == null)
        return NotFound();

    product.Name = updatedProduct.Name;
    product.Price = updatedProduct.Price;

    await _context.SaveChangesAsync();
    return NoContent();
}
```

```
// DELETE
[HttpDelete("{id}")]
public async Task<ActionResult> DeleteProduct(int id)
{
    var product = await _context.Products.FindAsync(id);

    if (product == null)
        return NotFound();

    _context.Products.Remove(product);
    await _context.SaveChangesAsync();

    return NoContent();
}
```

7. How This Works Internally

1. App Starts → EF Core creates a new InMemory DB

- Only inside application memory.
- No actual DB files.

2. When you call Add()

```
_context.Products.Add(product);
```

The entity is tracked in EF's memory store.

3. SaveChangesAsync()

```
await _context.SaveChangesAsync();
```

Commits changes into the InMemory storage.

4. No SQL is generated

Unlike SQL Server provider, InMemory uses its own internal collections.

5. Data lost on restart

Because it is held in RAM only.

8. Testing via Swagger

Common test examples:

Create Product (POST)

```
{  
  "name": "Laptop",  
  "price": 55000  
}
```

Get All Products (GET)

Returns list.

Update Product (PUT)

```
{  
  "name": "Gaming Laptop",  
  "price": 65000  
}
```

Delete Product (DELETE)

9. Complete Folder Structure

InMemoryApiDemo/

|

|— Controllers/

| └─ ProductsController.cs

```
|— Data/
|  |— AppDbContext.cs
|— Models/
|  |— Product.cs
|— Program.cs
|— InMemoryApiDemo.csproj
```

10. Summary

CRUD Operation EF Core InMemory

Create	Add + SaveChanges
Read	Find / ToList
Update	Modify entity + SaveChanges
Delete	Remove + SaveChanges
DB Persistence	No
Migrations	Not required
